# SRS · PayFlow Digital Wallet & Transaction Management System

Nov 1, 2026 · @SANTO

## 1. Introduction

This SRS specifies PayFlow v1.0 for Group 2: 38 functional requirements with Gherkin acceptance criteria, 16 business rules, the PostgreSQL schema, the REST API contract and 24 measurable non-functional requirements. It follows ISO/IEC/IEEE 29148:2018; requirement keywords follow RFC 2119 (MUST, SHOULD, MAY).

**Status.** Draft v0.1 for BA review. Business Analyst: Masum Billah. Scrum Master: Sourav Barman. The BA resolves section 7 before sign-off; the answers replace the proposed defaults marked *(default)*.

**Purpose.** One agreed, testable statement of what the wallet must do, so developers build and QA tests against the same list. Every requirement has an ID; ClickUp tasks and test cases cite those IDs.

### 1.1 References

| Ref | Document |
|---|---|
| R1 | PayFlow Digital Wallet & Transaction Management System — Project BRD |
| R2 | ISO/IEC/IEEE 29148:2018, Requirements engineering |
| R3 | RFC 2119, requirement keywords; RFC 9457, Problem Details for HTTP APIs |
| R4 | OWASP ASVS 4.0.3, Level 2 |
| R5 | IETF draft "The Idempotency-Key HTTP Header Field" |

## 1.2 Definitions

| Term | Meaning |
|---|---|
| Wallet | A simulated-BDT balance owned by one active customer, agent or merchant |
| Wallet number | The owner's registered 11-digit mobile number, used to address a wallet *(default, Q3)* |
| Transaction | One business operation (deposit, send, payment, cash-out, allocation) with a unique reference |
| Ledger entry | One signed movement on one wallet; every completed transaction writes exactly one debit and one credit |
| Treasury | The system-owned wallet that funds agent allocations; its balance is the negative of all money issued |
| Reference | Human-readable unique transaction ID, format `PF-YYYYMMDD-XXXXXXXX` |
| Idempotency key | Client-generated UUID sent on every money-moving request so a retry is processed once |
| Float | An agent's wallet balance, used for cash-in and received through cash-out |

## 1.3 User roles and permissions

Roles are fixed; each account holds exactly one role.

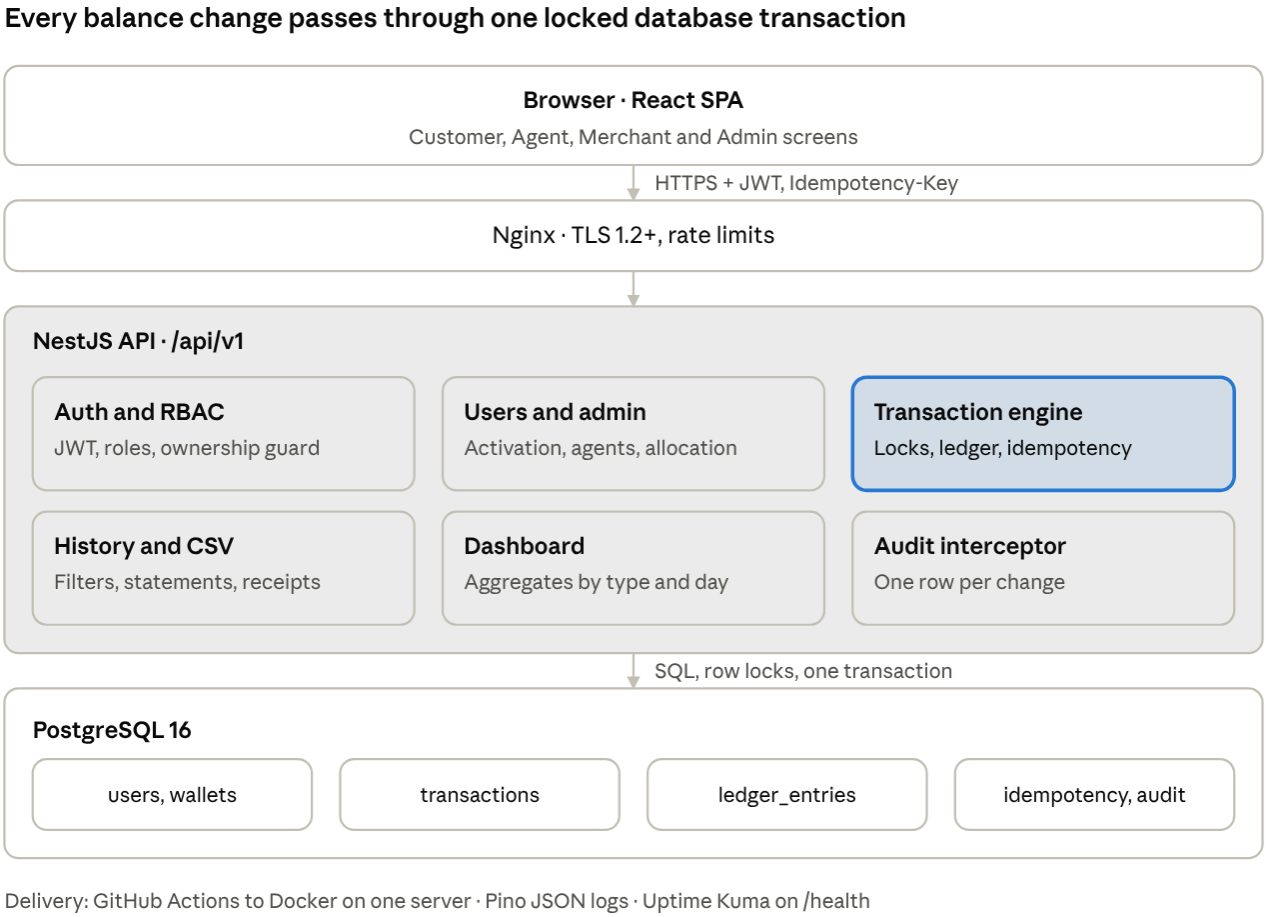
| Capability | Admin | Agent | Customer | Merchant |
|---|---|---|---|---|
| Register self (pending activation) | | | ✓ | ✓ |
| Log in, edit profile, change password | ✓ | ✓ | ✓ | ✓ |
| Activate or deactivate accounts; create agents | ✓ | | | |
| Allocate funds to agents | ✓ | | | |
| Agent Deposit (cash-in) | | ✓ | | |
| Initiate Cash-Out | | ✓ | | |
| Authorize or reject Cash-Out | | | ✓ | |

| Capability | Admin | Agent | Customer | Merchant |
|---|---|---|---|---|
| Send Money | | | ✓ | |
| Pay a merchant | | | ✓ | |
| Receive payments, verify by reference | | | | ✓ |
| Own history and CSV statement | | ✓ | ✓ | ✓ |
| Search any transaction, view audit trail, dashboard | ✓ | | | |

Administrators have no wallet and can never move money other than through FR-ADM-05 (allocation).

## 2. System overview and architecture

PayFlow is a three-tier web application whose only source of truth for money is a double-entry ledger in PostgreSQL. A balance changes only inside one database transaction that also writes the ledger entries, the transaction record and the audit row.




PayFlow architecture · 4 layers, 6 API modules

Only the transaction engine writes balances; every other module reads them or records what happened.

## 2.1 Reference stack

The team MAY substitute an equivalent tool in Sprint 1, provided the rules in 2.3 still hold.

| Layer | Choice | Why |
|---|---|---|
| Frontend | React 18, TypeScript, Vite, TanStack Query, React Hook Form + Zod | Typed forms; one validation schema shared with the API |
| Backend | Node.js 20 LTS, NestJS 10, TypeScript | Modules, guards and DTO validation out of the box |
| Data access | Prisma 5 with raw SQL for locking ( `SELECT … FOR UPDATE` ) | Migrations plus explicit row locks |
| Database | PostgreSQL 16, `READ COMMITTED` + row locks | ACID, CHECK constraints, partial unique indexes |
| API contract | OpenAPI 3.1, generated at `/api/docs` | One contract for frontend, QA and Postman |
| Auth | JWT access token (15 min) + rotating refresh token (7 days, httpOnly cookie); Argon2id | Short-lived access, revocable sessions |
| Testing | Jest + Supertest (API), Playwright (E2E), k6 (load, concurrency) | As named in the BRD |
| Delivery | Docker Compose, GitHub Actions, Nginx (TLS) | One-command local setup; CI gate on every pull request |
| Observability | Pino JSON logs with request ID, `/health` and `/ready`, Uptime Kuma | Basic monitoring per the BRD |

## 2.2 Money path (every money-moving request)

1. The client sends `POST /api/v1/transactions/{type}` with an `Idempotency-Key` header (UUID v4).
2. The API authenticates the JWT, checks the role and validates the body.
3. The idempotency store is checked: a completed key returns the stored response; a key still in progress returns 409.
4. `BEGIN`; both wallets are locked with `SELECT … FOR UPDATE` in ascending wallet-ID order, which prevents deadlocks.

5. Status, limits and balance are re-checked under the lock.
6. One `transactions` row, two `ledger_entries` rows (debit, credit) and one `audit_logs` row are inserted; both balances are updated.
7. `COMMIT`; the response is stored against the idempotency key and returned with the reference.
8. Any error before `COMMIT` rolls everything back; a FAILED transaction row is then written in a separate transaction for traceability.

## 2.3 Architecture rules

- Balances MUST change only in the transaction engine; no other module writes `wallets.balance`.
- Every rule in section 4 MUST be enforced on the server, and money rules also in the database; UI checks are for convenience only.
- Ledger, transaction and audit tables are append-only; the application database user has no `UPDATE` or `DELETE` on them.

## 2.4 Scope

| In scope (BRD §4) | Out of scope (BRD §4) |
|---|---|
| Registration, login, profile, password change | Real banks, payment gateways, real cash |
| Admin activation, deactivation, agent creation | KYC verification |
| Fund allocation from admin to agents | Mobile app |
| Agent Deposit (cash-in) | Multi-currency; BDT only, simulated |
| Send Money | Fees and cashback |
| Merchant Payment | Scheduled payments, international transfers |
| Cash-Out with customer authorization | Fraud detection or machine learning |
| History, filters, CSV statement | Notification services (SMS, email, push) |
| Admin search, audit trail, dashboard and reports | Reversal or refund of completed transactions *(default, Q6)* |
| CI/CD, logging, monitoring, documentation | |

### 2.5 Constraints and assumptions

- 12 weeks, 4 interns; one deployment server; simulated BDT with 2 decimal places.
- Seed data per BRD §9: 1 admin, 3 agents, ~20 customers, ~5 merchants with sample balances, all created through allocations and transfers so the ledger reconciles.
- All times are stored in UTC and shown in Asia/Dhaka (UTC+6).

## 3. Functional requirements

There are 38 requirements in six areas; 34 are MUST, 4 are SHOULD. Each has a user story and Gherkin acceptance criteria that QA automates as API or Playwright tests. Error codes refer to section 5.3.

| Area | IDs | Count |
|---|---|---|
| Authentication and profile | FR-AUTH-01 to 06 | 6 |
| Administration | FR-ADM-01 to 08 | 8 |
| Wallets | FR-WAL-01 to 03 | 3 |
| Transactions | FR-TXN-01 to 11 | 11 |
| History and statements | FR-HIS-01 to 05 | 5 |
| Dashboard and reports | FR-DSH-01 to 05 | 5 |

### 3.1 Authentication and profile

**FR-AUTH-01 · Self-registration · MUST**

As a customer or merchant, I want to register with my name, mobile, email and password, so that I can request a wallet.

```
Scenario: Valid registration creates a pending account
  Given no account uses mobile "01711000001" or email "rahim@example.com"
  When I register as CUSTOMER with name "Rahim", that mobile, that email and
password "Str0ng!Pass"
  Then the response is 201 with status "PENDING"
  And I cannot log in until an administrator activates the account

Scenario: Duplicate mobile is rejected
  Given an account already uses mobile "01711000001"
  When I register with the same mobile
  Then the response is 409 with code "MOBILE_TAKEN"
```

```
Scenario Outline: Invalid input is rejected
  When I register with <field> set to <value>
  Then the response is 422 naming <field>
  Examples:
    | field    | value          |
    | mobile   | "0171100"      |
    | email    | "rahim@"       |
    | password | "short1"       |
    | role     | "AGENT"        |
```

### FR-AUTH-02 · Login · MUST

As any user, I want to log in with mobile or email and password, so that I reach my role's screens.

```
Scenario: Active user logs in
  Given an ACTIVE account with mobile "01711000001"
  When I log in with the correct password
  Then I receive an access token valid for 15 minutes and a refresh cookie
valid for 7 days

Scenario: Pending or inactive user is refused
  Given an account with status "PENDING" or "INACTIVE"
  When I log in with the correct password
  Then the response is 403 with code "ACCOUNT_NOT_ACTIVE"

Scenario: Wrong credentials reveal nothing
  When I log in with an unknown mobile or a wrong password
  Then the response is 401 with the same message "Invalid credentials" in
both cases
```

### FR-AUTH-03 · Logout and session refresh · MUST

As a user, I want to log out and have my session renewed while I work, so that my wallet is not left open.

```
Scenario: Logout revokes the refresh token
  Given I am logged in
  When I log out
  Then my refresh token is revoked and a refresh attempt returns 401

Scenario: Deactivation ends sessions
  Given I am logged in
  When an administrator deactivates my account
  Then my next API call returns 401 within 15 minutes and refresh is refused
at once
```

### FR-AUTH-04 · Profile update · MUST

As a user, I want to update my name and email, so that my details stay correct.

```
Scenario: Name and email can change; mobile cannot
  When I update my name to "Rahim Uddin"
  Then the change is saved and written to the audit trail
  When I try to change my mobile
  Then the response is 422 with code "FIELD_IMMUTABLE"
```

### FR-AUTH-05 · Password change · MUST

As a user, I want to change my password, so that I can keep my account secure.

```
Scenario: Change requires the current password
  When I submit a wrong current password
  Then the response is 400 with code "CURRENT_PASSWORD_WRONG"

Scenario: Successful change signs out other sessions
  When I submit the correct current password and a new valid password
  Then the password is changed and every other refresh token of mine is
revoked
```

### FR-AUTH-06 · Failed-login lockout · SHOULD

As the platform, I want to slow repeated failed logins, so that passwords cannot be guessed.

```
Scenario: Five failures lock the account for 15 minutes
  Given 5 failed logins for one account within 15 minutes
  When a 6th attempt is made, even with the right password
  Then the response is 429 with code "TEMPORARILY_LOCKED" and a Retry-After
header
```

## 3.2 Administration

### FR-ADM-01 · Activate an account · MUST

As an administrator, I want to activate pending customers and merchants, so that only approved users get a wallet.

```
Scenario: Activation creates the wallet
  Given customer "Rahim" is PENDING
  When I activate him
  Then his status is ACTIVE, a wallet with balance 0.00 BDT exists and the
action is audited
```

### FR-ADM-02 · Deactivate and reactivate an account · MUST

As an administrator, I want to deactivate and reactivate any non-admin account, so that I can stop misuse without losing history.

```
Scenario: Deactivated wallets cannot send or receive
  Given Karim is INACTIVE
  When Rahim sends 100 BDT to Karim
  Then the response is 422 with code "RECIPIENT_INACTIVE" and no balance
changes

Scenario: Reason is required
  When I deactivate an account without a reason
  Then the response is 422 naming "reason"

Scenario: Balance is kept
  Given Karim has 2,000.00 BDT and is deactivated, then reactivated
  Then his balance is still 2,000.00 BDT
```

### FR-ADM-03 · Create an agent · MUST

As an administrator, I want to create agent accounts, so that agents are never self-registered.

```
Scenario: Admin-created agent is active with an empty wallet
  When I create agent "Agent Mirpur" with mobile "01811000001"
  Then the agent is ACTIVE with a 0.00 BDT wallet and must set a password on
first login
```

### FR-ADM-04 · Search users · MUST

As an administrator, I want to search users by name, mobile, role and status, so that I can find an account fast.

```
Scenario: Filtered, paginated user list
  Given 29 seeded users
  When I filter role CUSTOMER and status PENDING, page size 20
  Then I see only pending customers, newest first, with a total count
```

### FR-ADM-05 · Allocate funds to an agent · MUST

As an administrator, I want to allocate simulated funds to an agent, so that the agent can serve cash-in.

```
Scenario: Allocation moves money from the treasury
  Given agent "Agent Mirpur" has 0.00 BDT
  When I allocate 100,000.00 BDT with note "Opening float"
  Then the agent has 100,000.00 BDT, the treasury balance falls by 100,000.00
  And an ALLOCATION transaction with a reference and two ledger entries
exists

Scenario: Only agents can receive allocations
  When I allocate funds to a customer
  Then the response is 422 with code "NOT_AN_AGENT"
```

### FR-ADM-06 · Find a transaction by reference · MUST

As an administrator, I want to open any transaction by its reference, so that I can investigate disputes.

```
Scenario: Full investigation view
  When I search reference "PF-20261110-7Q2K9XAB"
  Then I see type, status, amount, both parties, both ledger entries with
balances after, initiator, timestamps and any failure reason
```

### FR-ADM-07 · View the audit trail · MUST

As an administrator, I want to filter the audit trail by actor, action, entity and date, so that I can see who did what and when.

```
Scenario: Activation is traceable
  Given admin "Sadia" activated Rahim at 10:02
  When I filter the audit trail by entity "Rahim"
  Then I see action ACCOUNT_ACTIVATED by "Sadia" at 10:02 with before and
after status
```

#### FR-ADM-08 · No balance edits, no deletes · MUST

As the platform owner, I want administrators unable to edit balances or delete transactions, so that the ledger stays trustworthy.

```
Scenario: No endpoint changes balances directly
  When any role calls PATCH on a wallet balance or DELETE on a transaction
  Then the response is 404 or 405 and nothing changes

Scenario: The database refuses tampering
  When the application user runs UPDATE or DELETE on ledger_entries
  Then PostgreSQL raises a permission error
```

### 3.3 Wallets

#### FR-WAL-01 · One wallet per account · MUST

As an active customer, agent or merchant, I want exactly one wallet that starts at 0 BDT, so that my money has one home.

```
Scenario: Second activation does not create a second wallet
  Given Rahim was activated, deactivated and activated again
  Then Rahim still owns exactly one wallet
```

#### FR-WAL-02 · View balance · MUST

As a wallet owner, I want to see my current balance, so that I know what I can spend.

```
Scenario: Balance matches the ledger
  When I open my home screen
  Then the balance equals the sum of my ledger entries, shown as "৳ 2,000.00"
```

#### FR-WAL-03 · Recipient preview · SHOULD

As a sender, I want to see the recipient's name before I confirm, so that I do not send to the wrong number.

```
Scenario: Masked name is shown
  When I enter wallet number "01711000002"
  Then I see "Karim H***" and the role CUSTOMER before the confirm button is
enabled
```

## 3.4 Transactions

### FR-TXN-01 · Agent Deposit (cash-in) · MUST

As an agent, I want to deposit to a customer's wallet from my float, so that customers can turn cash into wallet money.

```
Scenario: Successful cash-in
  Given agent float is 100,000.00 BDT and Rahim has 0.00 BDT
  When the agent deposits 5,000.00 BDT to "01711000001"
  Then the agent has 95,000.00 BDT, Rahim has 5,000.00 BDT
  And both see transaction type CASH_IN with the same reference

Scenario: Insufficient float
  Given agent float is 1,000.00 BDT
  When the agent deposits 5,000.00 BDT
  Then the response is 422 with code "INSUFFICIENT_BALANCE" and no balance
changes

Scenario: Recipient must be an active customer
  When the agent deposits to a merchant or an inactive customer
  Then the response is 422 with code "INVALID_RECIPIENT"
```

### FR-TXN-02 · Send Money · MUST

As a customer, I want to send money to another active customer, so that I can pay people without cash.

```
Scenario: Successful transfer
  Given Rahim has 5,000.00 BDT and Karim has 0.00 BDT
  When Rahim sends 3,000.00 BDT to Karim with note "Rent share"
  Then Rahim has 2,000.00 BDT, Karim has 3,000.00 BDT
  And both statements show the same reference

Scenario: Cannot send to self
  When Rahim sends to his own wallet number
  Then the response is 422 with code "SELF_TRANSFER"

Scenario: Recipient must be a customer
  When Rahim sends to an agent or merchant number
  Then the response is 422 with code "INVALID_RECIPIENT"
```

### FR-TXN-03 · Merchant Payment · MUST

As a customer, I want to pay an active merchant, so that I can buy goods without cash.

```
Scenario: Merchant sees the payment at once
  Given Rahim has 2,000.00 BDT
  When Rahim pays 1,200.00 BDT to merchant "Dhaka Bites"
  Then Rahim has 800.00 BDT, the merchant balance rises by 1,200.00
  And the payment with its reference is on the merchant's list within 2
seconds
```

### FR-TXN-04 · Cash-Out initiation · MUST

As an agent, I want to start a cash-out for a customer, so that the customer can turn wallet money into cash.

```
Scenario: Agent creates a pending request
  When the agent requests a 2,000.00 BDT cash-out for "01711000002"
  Then a CASH_OUT request with status PENDING_AUTHORIZATION and a 5-minute
expiry is created
  And no balance changes yet
```

### FR-TXN-05 · Cash-Out authorization · MUST

As a customer, I want to approve a cash-out from my own logged-in account, so that nobody withdraws my money without me.

```
Scenario: Customer authorizes with a password re-check
  Given Karim has 3,000.00 BDT and a pending 2,000.00 BDT request from the
agent
  When Karim authorizes it and re-enters his password
  Then Karim has 1,000.00 BDT, the agent float rises by 2,000.00, status is
COMPLETED

Scenario: Another customer cannot authorize
  When Rahim tries to authorize Karim's request
  Then the response is 404 and nothing changes
```

### FR-TXN-06 · Cash-Out rejection and expiry · MUST

As a customer, I want to reject or ignore a cash-out request, so that unwanted withdrawals never happen.

```
Scenario: Rejected request
  When Karim rejects the pending request
  Then its status is REJECTED and no balance changes

Scenario: Expired request
  Given a request created 5 minutes and 1 second ago
  When Karim tries to authorize it
  Then the response is 422 with code "REQUEST_EXPIRED" and its status is
EXPIRED
```

### FR-TXN-07 · Amount validation and limits · MUST

As the platform, I want amounts validated against fixed limits, so that bad or extreme values never reach the ledger.

```
Scenario Outline: Amount rules
  When a customer sends <amount> BDT
  Then the response is <status>
  Examples:
    | amount     | status                       |
    | 0          | 422 AMOUNT_INVALID           |
    | -50        | 422 AMOUNT_INVALID           |
    | 10.005     | 422 AMOUNT_INVALID           |
    | 9.99       | 422 AMOUNT_BELOW_MIN         |
    | 10.00      | 201                          |
    | 25000.01   | 422 AMOUNT_ABOVE_MAX         |
```

Default limits: minimum 10.00 BDT, maximum 25,000.00 BDT per transaction, for all types except allocation *(default, Q4)*.

### FR-TXN-08 · Unique transaction reference · MUST

As any party, I want every transaction to have one unique reference, so that we can talk about the same record.

```
Scenario: References never repeat
  When 10,000 transactions are created in a load test
  Then 10,000 distinct references matching "PF-YYYYMMDD-XXXXXXXX" exist
```

### FR-TXN-09 · Idempotent requests · MUST

As a user on a bad connection, I want a retried request processed once, so that I am never charged twice.

```
Scenario: Same key, same result
  Given Rahim sent 3,000.00 BDT with Idempotency-Key "6f1c…"
  When the identical request is sent again with the same key
  Then the response repeats the first one, with the same reference
  And Rahim was debited exactly once

Scenario: Same key, different body
  When a request reuses the key with amount 3,500.00
  Then the response is 422 with code "IDEMPOTENCY_KEY_REUSED"

Scenario: Missing key
  When a money-moving request has no Idempotency-Key
  Then the response is 400 with code "IDEMPOTENCY_KEY_REQUIRED"
```

#### FR-TXN-10 · Concurrency safety · MUST

As a wallet owner, I want simultaneous requests unable to spend the same money twice, so that my balance is never negative.

```
Scenario: Two 900 BDT sends from 1,000 BDT
  Given Rahim has 1,000.00 BDT
  When two sends of 900.00 BDT with different keys arrive at the same time
  Then exactly one succeeds and the other returns 422 "INSUFFICIENT_BALANCE"
  And Rahim ends with 100.00 BDT
```

#### FR-TXN-11 · Atomicity and failure record · MUST

As an administrator, I want a failed transfer to leave both wallets unchanged and still be recorded, so that disputes can be investigated.

```
Scenario: Fault in the middle rolls back
  Given a test hook fails the credit step
  When Rahim sends 500.00 BDT to Karim
  Then neither balance changes, no ledger entries exist for it
  And a FAILED transaction with reason "INTERNAL_ERROR" is visible to the
admin
```

### 3.5 History and statements

#### FR-HIS-01 · Transaction history · MUST

As a wallet owner, I want to see my transactions, so that I know where my money went.

```
Scenario: Each row has the BRD's fields
  When I open my history
  Then each row shows reference, date and time, type, sender, receiver,
amount, direction, status and my balance after
```

### FR-HIS-02 · Filters and pagination · MUST

As a wallet owner, I want to filter by date range and type, so that I can find a payment quickly.

```
Scenario: Filtered page
  When I filter type SEND_MONEY from 2026-11-01 to 2026-11-30, page size 20
  Then only matching rows appear, newest first, with total count and page
links

Scenario: Range limit
  When the range is longer than 366 days
  Then the response is 422 with code "RANGE_TOO_LONG"
```

### FR-HIS-03 · CSV statement · MUST

As a wallet owner, I want to download my filtered history as CSV, so that I can keep my own records.

```
Scenario: CSV matches the screen
  When I download the statement for November 2026
  Then I get a UTF-8 CSV named "payflow-statement-01711000001-2026-11.csv"
  And its rows, opening balance and closing balance match the screen
```

### FR-HIS-04 · Merchant verification by reference · MUST

As a merchant, I want to look up a payment by reference, so that I can confirm a customer has paid.

```
Scenario: Own payment found
  When I search a reference of a payment to me
  Then I see amount, payer's masked name, time and status COMPLETED

Scenario: Someone else's reference
  When I search a reference that is not a payment to me
  Then the response is 404
```

### FR-HIS-05 · Transaction receipt · SHOULD

As a wallet owner, I want a receipt screen after each transaction, so that I can show proof.

```
Scenario: Receipt after success
  When a send completes
  Then a receipt shows the reference, amount, recipient, time and new
balance, and can be printed
```

## 3.6 Dashboard and reports

### FR-DSH-01 · Platform counts · MUST

As an administrator, I want user counts and completed and failed transaction counts, so that I can see platform health.

```
Scenario: Counts match the seed data
  Given the seed data plus a known test run
  Then active users by role and completed and failed counts match the
expected values exactly
```

### FR-DSH-02 · Value by type · MUST

As an administrator, I want total value and count per transaction type, so that I know how the platform is used.

```
Scenario: Totals by type
  Then CASH_IN, SEND_MONEY, PAYMENT, CASH_OUT and ALLOCATION each show count
and total BDT for the period
```

### FR-DSH-03 · Daily and monthly trends · MUST

As an administrator, I want daily and monthly volume charts, so that I can see growth.

```
Scenario: Days with no activity show zero
  Then the daily chart for November 2026 has 30 points, with 0 on idle days
```

### FR-DSH-04 · Agent and funding summary · MUST

As an administrator, I want agent deposits, customer withdrawals and funding totals per agent, so that I can manage agent float.

```
Scenario: Per-agent figures reconcile
  Then for each agent, opening float + allocations + cash-out received −
cash-in given = current float
```

### FR-DSH-05 · Dashboard filters · SHOULD

As an administrator, I want to filter the dashboard by date range and type, so that I can answer specific questions.

```
Scenario: Filter applies to every widget
  When I set the range to Nov 1–15 and type PAYMENT
  Then every figure and chart on the page uses that range and type
```

## 4. Business rules and state machines

These 16 rules hold for every request; QA writes at least one negative test per rule, and the "Enforced by" column names the layer that guarantees it.

| ID | Rule | Enforced by |
|---|---|---|
| BR-01 | A wallet balance is never below 0.00; the treasury is the only wallet allowed below zero. | `CHECK (balance >= 0 OR kind = 'TREASURY')` + re-check under row lock |
| BR-02 | Every completed transaction writes exactly one debit and one credit of the same amount. | Transaction engine; reconciliation job |
| BR-03 | Sum of all wallet balances, treasury included, is always 0.00. User wallets therefore hold exactly what was allocated. | Nightly and post-test reconciliation query |
| BR-04 | A wallet's balance equals the sum of its ledger entries. | Reconciliation query; `balance_after` on each entry |
| BR-05 | Both parties of a transaction MUST be ACTIVE at commit time. | Status check under lock |
| BR-06 | Allowed pairs: allocation admin→agent; cash-in agent→customer; send customer→customer; payment customer→merchant; cash-out customer→agent. | Type-to-role table in the engine |
| BR-07 | Amounts are positive, have at most 2 decimals and lie within the configured limits. | DTO validation + `CHECK (amount > 0)` |

| ID | Rule | Enforced by |
|---|---|---|
| BR-08 | Wallets are locked in ascending wallet-ID order inside one database transaction. | Engine; deadlock test |
| BR-09 | Each money-moving request carries an idempotency key; one key produces at most one transaction per user. | `UNIQUE (user_id, key)` on `idempotency_keys` |
| BR-10 | Transaction references are unique and never reused. | `UNIQUE (reference)` |
| BR-11 | Ledger entries, transactions and audit logs are never updated or deleted, except a transaction's status from PENDING to a final state. | DB grants; status trigger |
| BR-12 | Only the customer who owns the wallet can authorize a cash-out on it, within 5 minutes of the request. | Ownership check; `expires_at` |
| BR-13 | Administrators cannot change balances except by allocation to an agent. | No endpoint; DB grants |
| BR-14 | Only administrators create agents; customers and merchants self-register as PENDING. | Registration DTO rejects other roles |
| BR-15 | Users see only their own wallet, transactions and statements; admins see all. | Ownership guard on every query |
| BR-16 | Every administrative action and every state change writes one audit row with actor, action, entity, before, after, IP and time. | Audit interceptor |

## 4.1 Account status

| From | Event | To | Who |
|---|---|---|---|
| — | Self-registration | PENDING | Customer, merchant |
| — | Agent created | ACTIVE | Admin |

| From | Event | To | Who |
|---|---|---|---|
| PENDING | Activate | ACTIVE (wallet created on first activation) | Admin |
| PENDING | Reject | REJECTED (final) | Admin |
| ACTIVE | Deactivate, reason required | INACTIVE | Admin |
| INACTIVE | Reactivate | ACTIVE | Admin |

### 4.2 Transaction status

| From | Event | To |
|---|---|---|
| — | Request accepted | PENDING (inside the DB transaction) |
| PENDING | Commit succeeds | COMPLETED (final) |
| PENDING | Validation or system error | FAILED (final), with `failure_code` |

A COMPLETED transaction is never changed. A correction is a new transaction, which needs the decision in Q6.

### 4.3 Cash-out request

| From | Event | To | Who |
|---|---|---|---|
| — | Agent initiates | PENDING_AUTHORIZATION, expires in 5 min | Agent |
| PENDING_AUTHORIZATION | Customer authorizes with password | COMPLETED (creates the CASH_OUT transaction) | Customer |
| PENDING_AUTHORIZATION | Customer rejects | REJECTED | Customer |
| PENDING_AUTHORIZATION | Agent cancels | CANCELLED | Agent |

| From | Event | To | Who |
|---|---|---|---|
| PENDING_AUTHORIZATION | 5 minutes pass | EXPIRED | System (checked on read and every minute) |

An agent MAY have at most 1 pending request per customer at a time.

# 5. Data contracts and interfaces

Nine tables carry the system; money is `NUMERIC(14,2)`, never a float, and every money table is append-only. The DDL below is the target of the first migration (Sprint 1).

## 5.1 Entities

| Entity | Key relationships | Notes |
|---|---|---|
| users | 1 → 0..1 wallets | Role, status, credentials |
| wallets | 1 → n ledger_entries | One per user; plus one TREASURY wallet with no user |
| transactions | 1 → 2 ledger_entries | One business operation |
| ledger_entries | n → 1 wallets, n → 1 transactions | Signed amount, balance after |
| cashout_requests | 0..1 → 1 transactions | Agent-initiated, customer-authorized |
| idempotency_keys | n → 1 users | Stored response for replay |
| refresh_tokens | n → 1 users | Rotating, revocable |
| audit_logs | n → 1 users (actor) | Append-only |
| settings | — | Limits, expiry minutes |

## 5.2 PostgreSQL DDL

```
CREATE TYPE user_role   AS ENUM ('ADMIN','AGENT','CUSTOMER','MERCHANT');
CREATE TYPE user_status AS ENUM ('PENDING','ACTIVE','INACTIVE','REJECTED');
CREATE TYPE wallet_kind AS ENUM ('USER','TREASURY');
CREATE TYPE txn_type    AS ENUM
('ALLOCATION','CASH_IN','SEND_MONEY','PAYMENT','CASH_OUT');
```

```
CREATE TYPE txn_status  AS ENUM ('PENDING','COMPLETED','FAILED');
CREATE TYPE co_status   AS ENUM
('PENDING_AUTHORIZATION','COMPLETED','REJECTED','CANCELLED','EXPIRED');

CREATE TABLE users (
  id              BIGSERIAL PRIMARY KEY,
  full_name       VARCHAR(120) NOT NULL,
  mobile          CHAR(11)     NOT NULL UNIQUE CHECK (mobile ~ '^01[3-9][0-9]
{8}$'),
  email           VARCHAR(254) NOT NULL UNIQUE,
  password_hash   TEXT         NOT NULL,
  role            user_role    NOT NULL,
  status          user_status  NOT NULL DEFAULT 'PENDING',
  must_reset_pw   BOOLEAN      NOT NULL DEFAULT FALSE,
  failed_logins   SMALLINT     NOT NULL DEFAULT 0,
  locked_until    TIMESTAMPTZ  NULL,
  status_reason   VARCHAR(255) NULL,
  created_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE wallets (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT        NULL UNIQUE REFERENCES users(id),
  kind        wallet_kind   NOT NULL DEFAULT 'USER',
  balance     NUMERIC(14,2) NOT NULL DEFAULT 0,
  version     INTEGER       NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
  CHECK (balance >= 0 OR kind = 'TREASURY'),
  CHECK ((kind = 'USER') = (user_id IS NOT NULL))
);
CREATE UNIQUE INDEX one_treasury ON wallets (kind) WHERE kind = 'TREASURY';

CREATE TABLE transactions (
  id               BIGSERIAL PRIMARY KEY,
  reference        CHAR(20)      NOT NULL UNIQUE,      -- PF-YYYYMMDD-
XXXXXXXX
  type             txn_type      NOT NULL,
  status           txn_status    NOT NULL DEFAULT 'PENDING',
  amount           NUMERIC(14,2) NOT NULL CHECK (amount > 0),
  from_wallet_id   BIGINT        NOT NULL REFERENCES wallets(id),
  to_wallet_id     BIGINT        NOT NULL REFERENCES wallets(id),
  initiated_by     BIGINT        NOT NULL REFERENCES users(id),
  note             VARCHAR(140)  NULL,
  failure_code     VARCHAR(40)   NULL,
  created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
```

```
  completed_at     TIMESTAMPTZ   NULL,
  CHECK (from_wallet_id <> to_wallet_id),
  CHECK ((status = 'FAILED') = (failure_code IS NOT NULL))
);
CREATE INDEX txn_from_created ON transactions (from_wallet_id, created_at
DESC);
CREATE INDEX txn_to_created   ON transactions (to_wallet_id, created_at
DESC);
CREATE INDEX txn_type_created ON transactions (type, created_at);

CREATE TABLE ledger_entries (
  id              BIGSERIAL PRIMARY KEY,
  transaction_id  BIGINT        NOT NULL REFERENCES transactions(id),
  wallet_id       BIGINT        NOT NULL REFERENCES wallets(id),
  amount          NUMERIC(14,2) NOT NULL CHECK (amount <> 0),  -- debit < 0,
credit > 0
  balance_after   NUMERIC(14,2) NOT NULL,
  created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
  UNIQUE (transaction_id, wallet_id)
);
CREATE INDEX ledger_wallet_created ON ledger_entries (wallet_id, created_at
DESC);

CREATE TABLE cashout_requests (
  id              BIGSERIAL PRIMARY KEY,
  agent_id        BIGINT        NOT NULL REFERENCES users(id),
  customer_id     BIGINT        NOT NULL REFERENCES users(id),
  amount          NUMERIC(14,2) NOT NULL CHECK (amount > 0),
  status          co_status     NOT NULL DEFAULT 'PENDING_AUTHORIZATION',
  transaction_id  BIGINT        NULL UNIQUE REFERENCES transactions(id),
  expires_at      TIMESTAMPTZ   NOT NULL,
  decided_at      TIMESTAMPTZ   NULL,
  created_at      TIMESTAMPTZ   NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX one_pending_cashout
  ON cashout_requests (agent_id, customer_id) WHERE status =
'PENDING_AUTHORIZATION';

CREATE TABLE idempotency_keys (
  user_id        BIGINT      NOT NULL REFERENCES users(id),
  key            UUID        NOT NULL,
  request_hash   CHAR(64)    NOT NULL,          -- SHA-256 of method + path +
body
  state          VARCHAR(12) NOT NULL CHECK (state IN
('IN_PROGRESS','DONE')),
  response_code  SMALLINT    NULL,
```

```
  response_body  JSONB       NULL,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, key)
);

CREATE TABLE refresh_tokens (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT      NOT NULL REFERENCES users(id),
  token_hash  CHAR(64)    NOT NULL UNIQUE,
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked_at  TIMESTAMPTZ NULL
);

CREATE TABLE audit_logs (
  id           BIGSERIAL PRIMARY KEY,
  actor_id     BIGINT       NULL REFERENCES users(id),   -- NULL = system job
  action       VARCHAR(60)  NOT NULL,
  entity_type  VARCHAR(40)  NOT NULL,
  entity_id    BIGINT       NOT NULL,
  before_data  JSONB        NULL,
  after_data   JSONB        NULL,
  ip_address   INET         NULL,
  request_id   UUID         NOT NULL,
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX audit_entity ON audit_logs (entity_type, entity_id, created_at
DESC);

CREATE TABLE settings (
  key    VARCHAR(60) PRIMARY KEY,
  value  JSONB       NOT NULL
);  -- min_amount 10.00, max_amount 25000.00, cashout_expiry_minutes 5

-- Append-only enforcement for the application role
REVOKE UPDATE, DELETE ON ledger_entries, audit_logs FROM payflow_app;
REVOKE DELETE ON transactions FROM payflow_app;
```

## 5.3 API contract

Base path `/api/v1`, JSON over HTTPS, bearer JWT except where marked public. Money is sent and returned as a decimal string (`"3000.00"`) to avoid float rounding. Lists take `page` (from 1) and `size` (1–100, default 20).

| Method | Path | Role | Purpose | FR |
|---|---|---|---|---|
| POST | /auth/register | Public | Customer or merchant sign-up | AUTH-01 |
| POST | /auth/login | Public | Access token + refresh cookie | AUTH-02 |
| POST | /auth/refresh | Cookie | Rotate tokens | AUTH-03 |
| POST | /auth/logout | Any | Revoke refresh token | AUTH-03 |
| GET, PATCH | /me | Any | Read or update profile | AUTH-04 |
| POST | /me/password | Any | Change password | AUTH-05 |
| GET | /admin/users | Admin | Search users | ADM-04 |
| POST | /admin/users/{id}/activate | Admin | Activate | ADM-01 |
| POST | /admin/users/{id}/deactivate | Admin | Deactivate with reason | ADM-02 |
| POST | /admin/users/{id}/reactivate | Admin | Reactivate | ADM-02 |
| POST | /admin/agents | Admin | Create agent | ADM-03 |
| POST | /admin/allocations | Admin | Allocate funds to an agent | ADM-05 |
| GET | /admin/transactions/{reference} | Admin | Investigation view | ADM-06 |
| GET | /admin/audit-logs | Admin | Filter audit trail | ADM-07 |
| GET | /admin/dashboard | Admin | KPIs, trends, agent summary | DSH-01 to 05 |

| Method | Path | Role | Purpose | FR |
|---|---|---|---|---|
| GET | /wallet | Wallet owner | Balance | WAL-02 |
| GET | /wallets/lookup?number= | Customer, agent | Masked recipient name | WAL-03 |
| POST | /transactions/cash-in | Agent | Agent Deposit | TXN-01 |
| POST | /transactions/send | Customer | Send Money | TXN-02 |
| POST | /transactions/payment | Customer | Merchant Payment | TXN-03 |
| POST | /cashout-requests | Agent | Initiate cash-out | TXN-04 |
| GET | /cashout-requests?status=PENDING_AUTHORIZATION | Customer, agent | Own pending requests | TXN-04 to 06 |
| POST | /cashout-requests/{id}/authorize | Customer | Authorize with password | TXN-05 |
| POST | /cashout-requests/{id}/reject | Customer | Reject | TXN-06 |
| POST | /cashout-requests/{id}/cancel | Agent | Cancel | TXN-06 |
| GET | /transactions | Wallet owner | History with filters | HIS-01, 02 |
| GET | /transactions/{reference} | Party to it | Receipt or merchant verification | HIS-04, 05 |
| GET | /transactions/statement.csv | Wallet owner | CSV statement | HIS-03 |
| GET | /health, /ready | Public | Liveness, DB readiness | NFR-OPS-02 |

Every `POST /transactions/*`, `/admin/allocations` and `/cashout-requests/{id}/authorize` call MUST carry `Idempotency-Key: <uuid>`.

**Send Money: request**

```
POST /api/v1/transactions/send
Idempotency-Key: 6f1c2b7e-3a0d-4f5e-9b1a-0c2d4e6f8a10
{
  "toWalletNumber": "01711000002",
  "amount": "3000.00",
  "note": "Rent share"
}
```

**Send Money: 201 response** (the same shape for every transaction type)

```
{
  "reference": "PF-20261110-7Q2K9XAB",
  "type": "SEND_MONEY",
  "status": "COMPLETED",
  "amount": "3000.00",
  "from": { "walletNumber": "01711000001", "name": "Rahim Uddin" },
  "to":   { "walletNumber": "01711000002", "name": "Karim H***" },
  "balanceAfter": "2000.00",
  "note": "Rent share",
  "createdAt": "2026-11-10T08:15:02.114Z",
  "completedAt": "2026-11-10T08:15:02.131Z"
}
```

**Error response** (RFC 9457 Problem Details, every 4xx and 5xx)

```
{
  "type": "https://payflow.local/errors/insufficient-balance",
  "title": "Insufficient balance",
  "status": 422,
  "code": "INSUFFICIENT_BALANCE",
  "detail": "Your balance is 1000.00 BDT; this transfer needs 1800.00 BDT.",
  "requestId": "0b6c9f0e-6d1e-4c51-8a7d-2f3b9a1c4e55",
  "errors": []
}
```

| HTTP | Codes |
|---|---|
| 400 | VALIDATION_FAILED, IDEMPOTENCY_KEY_REQUIRED, CURRENT_PASSWORD_WRONG |
| 401 | UNAUTHENTICATED, INVALID_CREDENTIALS, TOKEN_EXPIRED |
| 403 | FORBIDDEN_ROLE, ACCOUNT_NOT_ACTIVE |

| HTTP | Codes |
|---|---|
| 404 | NOT_FOUND (also used for another user's resource, so existence is not revealed) |
| 409 | MOBILE_TAKEN, EMAIL_TAKEN, IDEMPOTENCY_IN_PROGRESS, PENDING_REQUEST_EXISTS |
| 422 | INSUFFICIENT_BALANCE, INVALID_RECIPIENT, RECIPIENT_INACTIVE, SELF_TRANSFER, AMOUNT_INVALID, AMOUNT_BELOW_MIN, AMOUNT_ABOVE_MAX, REQUEST_EXPIRED, IDEMPOTENCY_KEY_REUSED, NOT_AN_AGENT, FIELD_IMMUTABLE, RANGE_TOO_LONG |
| 429 | RATE_LIMITED, TEMPORARILY_LOCKED |
| 500 | INTERNAL_ERROR (no stack trace in the body) |

## 5.4 External interfaces

| Type | Requirement |
|---|---|
| User interface | Responsive from 360 px to 1440 px; role home screens: Customer (balance, send, pay, pending cash-outs, history), Agent (float, cash-in, cash-out requests, history), Merchant (payments received, verify reference), Admin (users, agents, allocations, transactions, audit, dashboard). Amounts shown as ৳ with thousands separators. WCAG 2.1 AA contrast and keyboard access. |
| Hardware | None. No card readers, printers or biometric devices; receipts print through the browser. |
| Software | PostgreSQL 16; current Chrome, Firefox, Edge and Safari (last 2 versions); OpenAPI 3.1 document for Postman and generated clients. No external payment, SMS or email service. |
| Files | CSV statements per RFC 4180, UTF-8 with BOM so Excel shows ৳ correctly; columns: reference, date_time_dhaka, type, direction, counterparty, amount, status, balance_after. |

# 6. Non-functional requirements

Each of the 24 NFRs has a number and a test; thresholds are measured on the staging server (2 vCPU, 4 GB RAM, PostgreSQL on the same host) with the BRD seed data plus 50,000 generated transactions.

## 6.1 Performance and capacity

| ID | Requirement | Threshold | Verified by |
|---|---|---|---|
| NFR-PERF-01 | Money-moving API latency | p95 ⩽ 300 ms, p99 ⩽ 800 ms at 50 requests/s | k6, 10-minute constant-rate run |
| NFR-PERF-02 | Read API latency (balance, history page, lookup) | p95 ⩽ 200 ms at 100 requests/s | k6 |
| NFR-PERF-03 | Concurrent users | 200 virtual users for 10 minutes with error rate < 1% (business 422s excluded) | k6 ramp test |
| NFR-PERF-04 | Hot-wallet contention | 50 parallel sends from one wallet: 0 negative balances, 0 deadlock errors returned to clients | k6 + reconciliation query |
| NFR-PERF-05 | CSV statement | 12 months / 5,000 rows generated in ⩽ 3 s | API test |
| NFR-PERF-06 | Dashboard | Loads in ⩽ 2 s for a 31-day range | Playwright timing |
| NFR-PERF-07 | Frontend | Largest Contentful Paint ⩽ 2.5 s on simulated Fast 4G; JS bundle ⩽ 300 KB gzipped | Lighthouse CI |

## 6.2 Security

| ID | Requirement | Threshold | Verified by |
|---|---|---|---|
| NFR-SEC-01 | Password storage | Argon2id, memory 19 MiB, 2 iterations; passwords 8–64 characters with a letter and a digit, checked against a top-10,000 breached list | Unit test, code review |
| NFR-SEC-02 | Tokens | Access JWT 15 min, HS256 secret ≥ 32 bytes from env; refresh token 7 days, rotated on use, stored as SHA-256 hash; reuse of a rotated token revokes the whole session family | API tests |
| NFR-SEC-03 | Authorization | Role guard + ownership check on 100% of endpoints; a generated matrix test calls every endpoint with every role | API regression suite |
| NFR-SEC-04 | Object-level access | Changing an ID or reference in any request never returns another user's data (404) | ID-tampering tests on every `{id}` route |
| NFR-SEC-05 | Transport | HTTPS only, TLS 1.2+, HSTS 6 months; cookies `HttpOnly; Secure; SameSite=Strict` | SSL Labs grade A, header check |
| NFR-SEC-06 | Input and injection | All input validated by DTO schema; parameterised SQL only; CSP blocks inline scripts | OWASP ZAP baseline scan, 0 high findings |
| NFR-SEC-07 | Rate limiting | Login 10/min per IP; money-moving 30/min per user; other API 300/min per user | API tests return 429 |
| NFR-SEC-08 | Secrets and logs | No secret in the repository; passwords, tokens and full JWTs never logged | gitleaks in CI; log review |

## 6.3 Availability, reliability and data integrity

| ID | Requirement | Threshold | Verified by |
|---|---|---|---|
| NFR-REL-01 | Availability | 99.5% monthly during the demo period (⩽ 3 h 39 min downtime) | Uptime Kuma, 1-minute checks |
| NFR-REL-02 | Ledger reconciliation | BR-03 and BR-04 hold with 0 differences after every test run and nightly at 02:00 | Reconciliation job; CI step |
| NFR-REL-03 | Atomicity under faults | 100% of fault-injection cases leave both wallets unchanged | Fault-injection tests |
| NFR-REL-04 | Backups | Nightly `pg_dump`, 7 daily + 4 weekly copies; RPO ⩽ 24 h, RTO ⩽ 2 h; one restore drilled before the final demo | Restore drill log |
| NFR-REL-05 | Graceful failure | DB unavailable → 503 within 5 s, no partial writes; app recovers without restart once the DB returns | Chaos test (stop DB container) |

### 6.4 Operability and maintainability

| ID | Requirement | Threshold | Verified by |
|---|---|---|---|
| NFR-OPS-01 | CI/CD | Every pull request runs lint, unit, API and E2E smoke tests; main deploys to staging automatically in ⩽ 15 min | GitHub Actions |
| NFR-OPS-02 | Monitoring and logs | JSON logs with request ID, user ID, route, status, duration; `/health` and `/ready` endpoints; alert after 2 failed checks | Log review, uptime monitor |
| NFR-OPS-03 | Code quality | Backend unit coverage ⩾ 80% on the transaction engine, ⩾ 60% overall; 0 ESLint errors | CI coverage gate |
| NFR-OPS-04 | Documentation | OpenAPI document matches the code (contract test); README sets up locally in ⩽ 15 min; user guide per role | Teammate setup check; demo |

## 7. Structural gap analysis and interview questions

The BRD leaves 12 points open; 4 are high risk because they decide how money leaves the system. Until the BA answers, the SRS uses the proposed default, marked *(default)* where it appears.

| # | BRD ref | Gap | Risk | Proposed default |
|---|---|---|---|---|
| Q1 | §4, §5 | Merchants receive payments, but no flow lets money leave a merchant wallet. | High: merchant balances grow forever; BR-03 still holds but the demo looks broken | Merchants MAY cash out through an agent, same flow as customers |
| Q2 | §4 step 2 | Allocation to agents is defined; returning or reclaiming agent float is not. | High: no way to correct an over-allocation without editing balances | Admin MAY record a REVERSE_ALLOCATION (agent → treasury), same controls |

| # | BRD ref | Gap | Risk | Proposed default |
|---|---|---|---|---|
| Q3 | §4 step 3 | The "customer's wallet number" is not defined. | Medium | Wallet number = registered mobile |
| Q4 | §6 | "Valid amount" has no bounds; no daily limits. | Medium: k6 tests and validation need fixed numbers | Min 10.00, max 25,000.00 BDT per transaction; no daily limit |
| Q5 | §4 step 6 | How the customer authorizes a cash-out is not defined. | High: weak authorization means an agent can withdraw for a customer | Customer approves in their own session and re-enters the password; request expires in 5 min |
| Q6 | §3, §6 | Admins investigate disputes but cannot correct them, and may not edit balances. | High: a wrong transfer can never be fixed | No reversals in v1.0; a dispute is recorded as an audit note only |
| Q7 | §6 | Password reset is not mentioned, and notifications are out of scope. | Medium: locked-out users cannot recover | Admin triggers a reset; the user sets a new password on next login |
| Q8 | §4 step 1 | Can an admin reject a pending registration, or only activate? | Low | Admin can reject; status REJECTED is final |
| Q9 | §6 | What happens to pending cash-out requests when either party is deactivated? | Medium | They are cancelled automatically at deactivation |
| Q10 | §4 step 1 | Merchants register with the same fields as customers; no business name. | Low: payment screens need a shop name | Add `business_name` (required for merchants) |
| Q11 | §6 | "Funding summaries" on the dashboard are not defined. | Low | Total allocated, total float held by agents, total in customer and merchant wallets |

| # | BRD ref | Gap | Risk | Proposed default |
|---|---|---|---|---|
| Q12 | §2 | Project owner name and PIN are placeholders. | Low: nobody named to accept the release | The BA, Masum Billah, acts as owner until named |

### 7.1 Questions for the project owner

1. Can a merchant move money out of their wallet, and if so through an agent cash-out or another flow?
2. Can an administrator take float back from an agent, and must that use the same recorded process as allocation?
3. Is a wallet identified by the owner's mobile number, or by a separate wallet number?
4. What are the minimum and maximum amounts per transaction, and are there daily limits per user or per agent?
5. How does a customer authorize a cash-out: password, a separate PIN, or approval in their session only? How long is a request valid?
6. When a transfer was made by mistake, is any correction allowed, and who may approve it?
7. How does a user recover a forgotten password without SMS or email?
8. May an administrator reject a pending registration instead of activating it?
9. When an account is deactivated, should its pending cash-out requests be cancelled?
10. Do merchants need a business name and category at registration?
11. Which figures make up the "funding summary" on the admin dashboard?
12. Who is the project owner who accepts the final release?

## 8. Traceability to the BRD

Every BRD success criterion and demo scenario maps to at least one requirement and one automated test. ClickUp tasks (WAL-xx) are mapped to these IDs after sign-off.

### 8.1 Success criteria

| BRD criterion (§10) | Requirements | Test |
|---|---|---|
| Balance accuracy | BR-01, BR-03, BR-04, NFR-REL-02 | Reconciliation query after every CI run |
| Atomicity | FR-TXN-11, NFR-REL-03 | Fault-injection API tests |

| BRD criterion (§10) | Requirements | Test |
|---|---|---|
| Concurrency (2 × 900 from 1,000) | FR-TXN-10, BR-08, NFR-PERF-04 | k6 parallel scenario |
| Idempotency | FR-TXN-09, BR-09 | Replay the same key 5 times |
| Validation | FR-TXN-01 to 03, FR-TXN-07, BR-05 to BR-07 | Negative API cases |
| Authorization | Section 1.3, BR-15, NFR-SEC-03, NFR-SEC-04 | Role matrix and ID-tampering suite |
| Traceability | FR-TXN-08, FR-ADM-07, FR-ADM-08, BR-11, BR-16 | Audit review; DB permission test |
| Reporting | FR-HIS-01 to 03, FR-DSH-01 to 05 | Compare against the seed dataset |
| Delivery | NFR-OPS-01 to 04 | Final demo checklist |

## 8.2 Demo scenarios

| BRD scenario | Requirements | Expected result |
|---|---|---|
| Agent deposits 5,000 BDT to Rahim | FR-TXN-01 | Agent −5,000, Rahim +5,000, one reference on both histories |
| Rahim sends 3,000 BDT to Karim | FR-TXN-02, FR-TXN-08 | Rahim 2,000.00, Karim 3,000.00, same reference |
| Rahim pays 1,200 BDT to a restaurant | FR-TXN-03, FR-HIS-04 | Merchant sees and verifies the reference |
| Karim cashes out 2,000 BDT via agent | FR-TXN-04, FR-TXN-05 | Karim debited after his authorization, agent credited |
| Two browsers send 900 BDT from 1,000 BDT | FR-TXN-10 | One success, one 422, balance 100.00 |
| Same transfer submitted twice | FR-TXN-09 | Processed once, same reference returned |

## 8.3 Version history

| Version | Date | Change | By |
|---|---|---|---|
| 0.1 | Nov 1, 2026 | First draft from the BRD | Claude, for Md. Rabiul Islam |
| 1.0 | — | Sign-off after section 7 is answered | Masum Billah (BA), pending |